

Django Unit Test Documentation

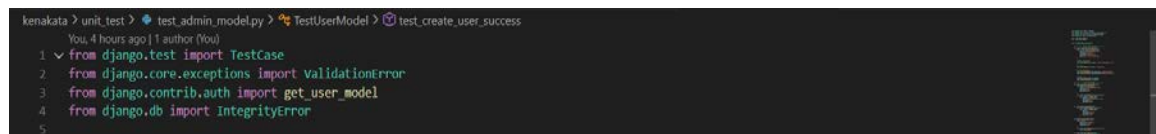
Unittest is a **built-in Python testing framework** used in Django to create automated tests. It allows you to test models, views, forms, and other components to ensure they behave correctly.

Key points:

- Each test checks a **single functionality** (test case).
- Multiple tests can be grouped into a **test suite**.
- Django extends unittest via `django.test.TestCase`.

How to Create a Unit Test in Django

1. Import necessary classes:

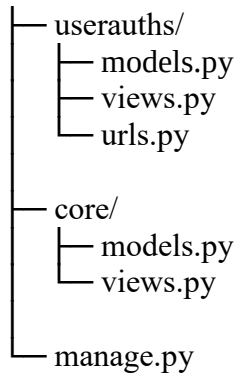


```
kenakata > unit_test > test_admin_model.py > TestUserModel > test_create_user_success
You, 4 hours ago | 1 author (You)
1 from django.test import TestCase
2 from django.core.exceptions import ValidationError
3 from django.contrib.auth import get_user_model
4 from django.db import IntegrityError
5
```

2. **Create a test class** inheriting from `TestCase`.
3. **Set up test data** in the `setUp()` method.
4. **Write test methods** (method names must start with `test_`).
5. **Use assertions** to check expected outcomes, e.g., `assertEqual`, `assertTrue`, `assertContains`.

Folder Structure

```
kenakata/
├── unit_tests/
│   ├── __init__.py
│   ├── test_admin_model.py
│   ├── test_login.py
│   └── test_my_profile.py
```



Example: Testing a Django Model

test_admin_model.py

```
9 class TestUserModel(TestCase):
10
11     # ----- BASIC USER CREATION -----
12     def test_create_user_success(self):
13         user = User.objects.create_user(
14             email="TestUser@example.com",
15             username="testuser",
16             password="mypassword123",
17             bio="Hello! I am a test user."
18         )
19
20         # Email normalized
21         self.assertEqual(user.email, "testuser@example.com")
22
23         # _str_
24         self.assertEqual(str(user), "testuser")
25
26         # Password hashing
27         self.assertNotEqual(user.password, "mypassword123")
28         self.assertTrue(user.check_password("mypassword123"))
29
30         # Flags
31         self.assertFalse(user.is_staff)
32         self.assertTrue(user.is_active)
33
34     # ----- FIELD VALIDATIONS -----
35     def test_email_must_be_unique(self):
36         User.objects.create_user(
37             email="duplicate@example.com",
38             username="one",
39             password="pass"
40         )
41
42         with self.assertRaises(IntegrityError):
43             User.objects.create_user(
```

test_login.py

```
kenakata / unit_test / test_login.py / ...
Newsin-14, 2 hours ago | 1 author (Newsin-14)

8 class TestLoginView(TestCase):
9
10     def setUp(self):
11         # Create normal user
12         self.user = User.objects.create_user(
13             email="user@example.com",
14             username="testuser",
15             password="testpass"
16         )
17
18         # Create staff user (admin)
19         self.admin = User.objects.create_user(
20             email="admin@example.com",
21             username="adminuser",
22             password="adminpass",
23             is_staff=True
24         )
25
26         # Create superuser
27         self.superuser = User.objects.create_superuser(
28             email="super@example.com",
29             username="superuser",
30             password="superpass"
31         )
32
33         self.client = Client()
34
35
36     def test_login_view_success(self):
37         response = self.client.post(reverse("login"), {
38             "email": "user@example.com",
39             "password": "testpass"
40         }, follow=True)
41
42         self.assertEqual(response.status_code, 200)
43         self.assertContains(response, "Welcome back, testuser!")
```

test_my_profile.py

```
Farhadsam321, 2 hours ago | 1 author (Farhadsam321)
Farhadsam321, 2 hours ago • added unit test ...

1 from django.test import TestCase, Client
2 from django.urls import reverse
3 from django.contrib.auth import get_user_model
4 from core.models import Order, Wishlist, Review, Address, PaymentMethod, Product, OrderItem
5
6 User = get_user_model()
7
8
9 Farhadsam321, 2 hours ago | 1 author (Farhadsam321)
10 class TestAccountView(TestCase):
11
12     def setUp(self):
13         # Create user
14         self.user = User.objects.create_user(
15             (parameter) password: str | None
16             password="testpass"
17         )
18
19         # login client
20         self.client = Client()
21         self.client.login(email="user@example.com", password="testpass")
22
23         # Products
24         self.product1 = Product.objects.create(title="Product1", price=100)
25         self.product2 = Product.objects.create(title="Product2", price=200)
26
27         # Order and items
28         self.order = Order.objects.create(user=self.user, total_amount=300)
29         self.order_item1 = OrderItem.objects.create(order=self.order, product=self.product1, quantity=1, price=100)
30         self.order_item2 = OrderItem.objects.create(order=self.order, product=self.product2, quantity=1, price=200)
31
32         # Wishlist
33         self.wishlist = Wishlist.objects.create(user=self.user)
34         self.wishlist.products.add(self.product1)
35
```

Running Unit Tests

Run **all tests** in your project:

```
python manage.py test
```

Run tests from a **specific folder**:

```
python manage.py test unit_tests
```

Generate coverage report:

```
coverage run --source="unit_test" manage.py test
```

```
coverage report
```

Test Result Screenshots

Coverage report: 96%				
FilesFunctionsClasses				
coverage.py v7.12.0, created at 2025-12-02 23:57 +0600				
File	statements	missing	excluded	coverage ▾
unit_test \ Click to sort	0	0	0	100%
unit_test \ test_my_profile.py	65	0	0	100%
unit_test \ test_login.py	60	2	0	97%
unit_test \ test_admin_model.py	58	6	0	90%
Total	183	8	0	96%
coverage.py v7.12.0, created at 2025-12-02 23:57 +0600				

